

4.1.1 BNF 定義と CF 文法

¹ 学生実験 c において対象とする原始プログラムを記述するプログラミング言語

```

procedure main(x,y):
  x:=y+1

```

BNF 定義

形式的にはプログラミング言語は文脈自由文法で定義されるが、文脈自由文法では生成規則の集合を記述すれば十分であることが多い。このため、生成規則の集合でプログラミング言語を定義する。その際に一般にBNF(Backus-Nauer Form) 定義と呼ばれる記法が用いられる。BNF は生成規則とほぼ同じであるが、 $\rightarrow$ の代わりに $::=$ を用いる。また、同じ左辺をもつ規則をまとめて、右辺を $|$ で並べて記載する。 $\langle \text{proc_decl} \rangle$ についてのBNF 定義は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &::= \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\quad | \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 \langle \text{proc_name} \rangle &::= \text{'IDENT'} \\
 \langle \text{id_list} \rangle &::= \text{'IDENT'} \\
 &\quad | \langle \text{id_list} \rangle \text{' , ' } \text{'IDENT'}
 \end{aligned}$$

最左導出と最右導出

導出の途中に現れる系列 $\alpha \in (N \cup \Sigma)^*$ において、 α が複数の非終端記号を含むときどの非終端記号を置き換えて導出するかによって、複数の導出が存在する。例えば、 A, B を非終端記号に含み、 $\{a, b, c\} \subseteq \Sigma$, $A \rightarrow a$, $B \rightarrow b$ という生成規則を持つとき、 ABc に対する導出は $ABc \Rightarrow aBc$ と $ABc \Rightarrow Abc$ がある。 $\alpha \in (N \cup \Sigma)^*$ に対する導出のうち、最も左にある非終端記号を置き換える導出を**最左導出**、最も右にある非終端記号を置き換える導出を**最右導出**といい、それぞれ $\alpha \Rightarrow_{lm} \beta$, $\alpha \Rightarrow_{rm} \beta'$ のように記述する。

α の $A \rightarrow \gamma$ による最左導出 $\alpha \Rightarrow_{lm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_1 \in \Sigma^*$ である。すなわち、 α の A より左の α_1 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

α の $A \rightarrow \gamma$ による最右導出 $\alpha \Rightarrow_{rm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_2 \in \Sigma^*$ である。すなわち、 α の A より右の α_2 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

最左導出、最右導出の0回以上の繰り返しをそれぞれ $\Rightarrow_{lm}^*$, $\Rightarrow_{rm}^*$ で表す。

例 4.2. $PL2$ の $\langle \text{proc_decl} \rangle$ からの最左導出は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &\Rightarrow_{lm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' , ' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle
 \end{aligned}$$

さらに $\langle \text{inblock} \rangle \Rightarrow_{lm}^* \text{'IDENT' ':' 'IDENT' '+' 'NUMBER'}$ であるとき

$$\Rightarrow_{lm}^* \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':' 'IDENT' '+' 'NUMBER'}$$

$PL2$ の $\langle \text{proc_decl} \rangle$ からの最右導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 ここで $\langle \text{inblock} \rangle \Rightarrow_{rm}^* \text{'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$ であるとき
 $\Rightarrow_{rm}^* \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

開始記号 S からの 0 回以上の導出によって得られる $\alpha \in (N \cup \Sigma)^*$ (ここで, $S \Rightarrow^* \alpha$) のことを **文形式** (sentential form) という. さらに, $S \Rightarrow_{lm}^* \beta$, $S \Rightarrow_{rm}^* st\gamma$ となる $\beta, \gamma \in (N \cup \Sigma)^*$ のことをそれぞれ左文形式, 右文形式という.

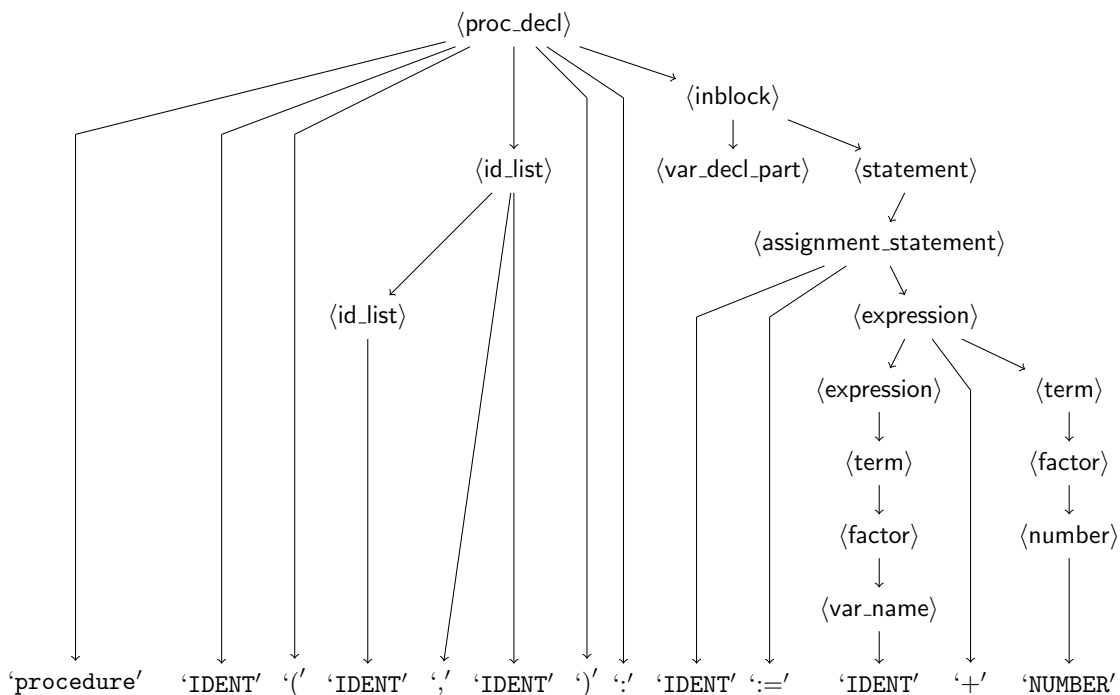
4.1.2 解析木

導出の過程は木構造で表現することができる. 非終端記号, 終端記号を頂点として表し, $A \rightarrow \alpha$ で導出したとき, A の頂点から $\alpha = \alpha_1\alpha_2\cdots\alpha_n$ ($\alpha_i \in N \cup \Sigma$) の α_i に有向辺を追加する. S からの導出をこのようにして表すと, S の頂点を根, $a \in \Sigma$ の頂点を葉とする木構造が構成される. この木構造のことを**解析木** (parse tree)(あるいは**構文木** (syntax tree)) という.

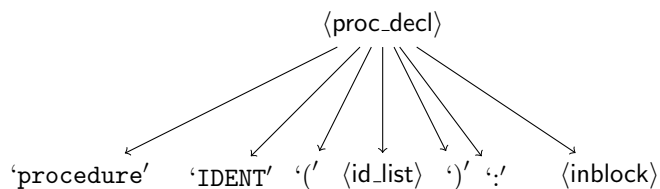
例 4.1 の例の (最左) 導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{var_decl_part} \rangle \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{assignment_statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{term} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{factor} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{var_name} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{factor} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{number} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

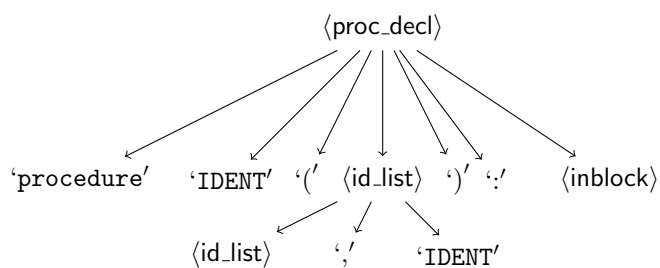
この導出から構成される木構造は以下のようになる.



解析木を構成するにあたって、最左導出では、開始記号は単一の記号なので、必ず最左の非終端記号であり、以下のような木を構成する。

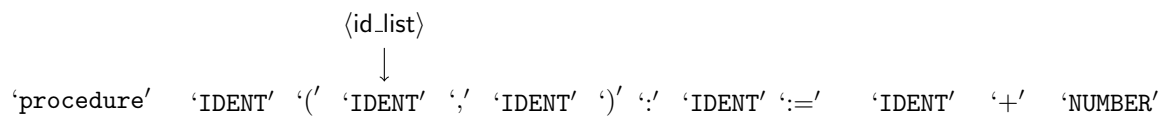


次に最左の非終端記号である <id_list> を導出すると以下の木が得られる。

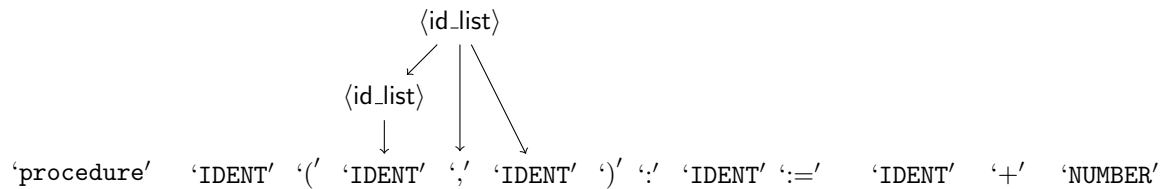


このように最左導出によって構文木は根から下向きに構成される。このような構文解析を**下降型構文解析**という。

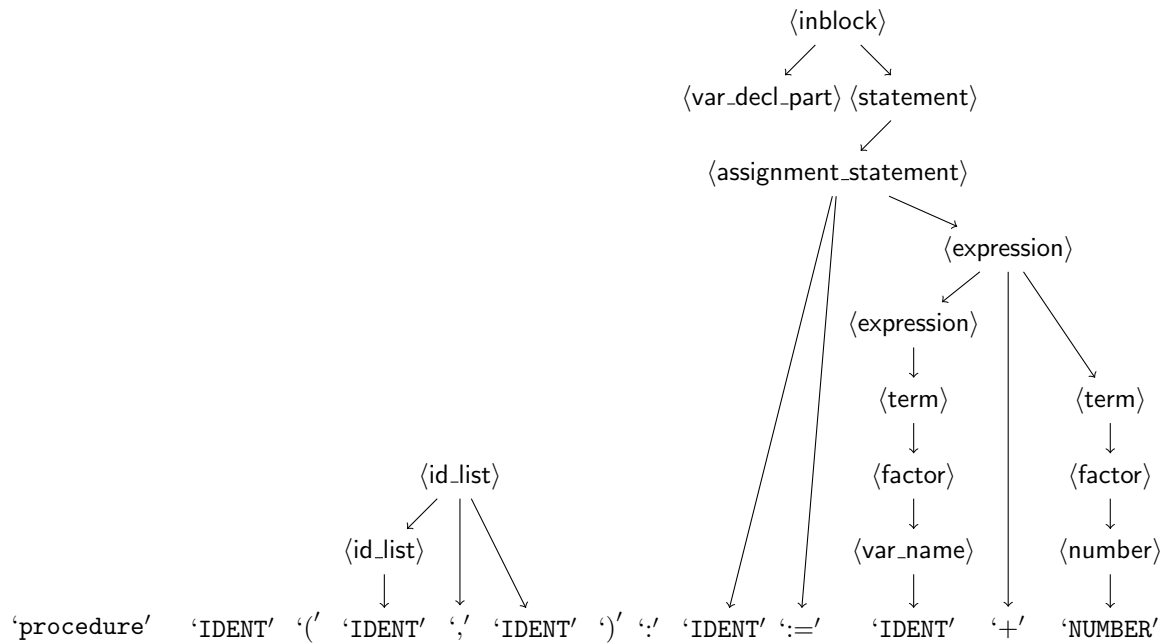
入力系列に対して引数リスト <id_list> を構成する 'IDENT' に <id_list> → 'IDENT' を逆向きに適用し、木を構成する。



次に, $\langle \text{id_list} \rangle \rightarrow \langle \text{id_list} \rangle \text{'IDENT'}$ を逆向きに適用する.



以下, 右辺にマッチする生成規則を適用し, $\langle \text{inblock} \rangle$ の部分木を生成する.



最後に $\langle \text{proc_decl} \rangle$ を左辺に持つ規則によって木が構成される.

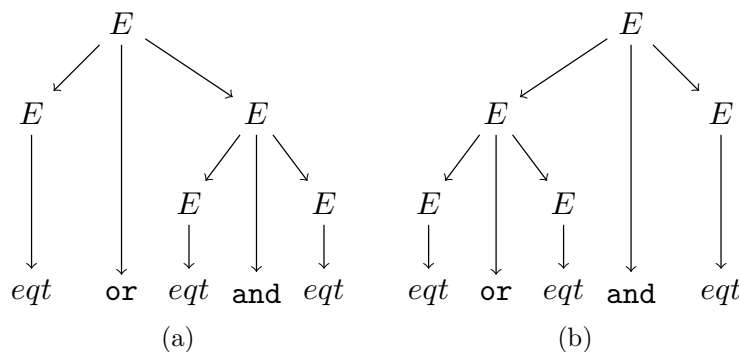


図 4.1: 複数の解析木

$L(G_1)$ に属する $eqt \text{ or } eqt \text{ and } eqt$ という系列に対して図 4.1 のような 2 つの解析木が存在する。導出において、 $\underline{E}$ を置き換えの対象となった E とすると、(a) は $\underline{E} \Rightarrow \underline{E} \text{ or } E \Rightarrow eqt \text{ or } \underline{E} \Rightarrow eqt \text{ or } \underline{E} \text{ and } E \Rightarrow eqt \text{ or } eqt \text{ and } \underline{E} \Rightarrow eqt \text{ or } eqt \text{ and } eqt$, (b) は $\underline{E} \Rightarrow \underline{E} \text{ and } E \Rightarrow \underline{E} \text{ or } E \text{ and } E \Rightarrow eqt \text{ or } \underline{E} \text{ and } E \Rightarrow eqt \text{ or } eqt \text{ and } \underline{E} \Rightarrow eqt \text{ or } eqt \text{ and } eqt$, という導出に対する解析木となる。

このように 1 つのトークン系列に対して 2 つ以上の異なる解析木が存在するとき文法は曖昧であるという。

3 つの eqt のトークンが具体的にそれぞれ $x = 1, y = 2, z = 3$ であるとする。 x, y, z がすべて 1 であるとする、 $x = 1$ は真、 $y = 2, z = 3$ は偽となる。 and, or がそれぞれ論理積、論理和を表すとする、(a) の解釈では、最初の $eqt(x = 1)$ が真となるので、全体の論理式は真であるが、(b) の解釈では最後の $eqt(z = 3)$ が偽であるので論理式は偽となる。

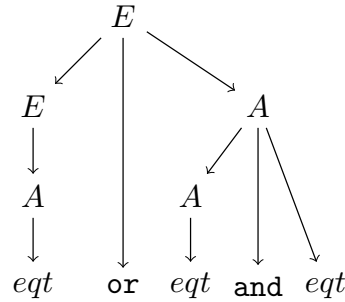
曖昧な文法では入力トークンの系列の意味が定まらないため、プログラミング言語の文法は曖昧でないことが望まれる。

論理和と論理積の場合、一般には和が積が優先されると解釈されるため、(a) の解釈をとる。これにそって G_1 の曖昧さを除去した文法 G_2 を以下に示す。

$$G_2 = \langle \{E, A\}, \{\text{or}, \text{and}, eqt\}, P_2, E \rangle$$

$$\text{ここで, } P_2 = \left\{ \begin{array}{l} E \rightarrow E \text{ or } A \\ E \rightarrow A \\ A \rightarrow A \text{ and } eqt \\ A \rightarrow eqt \end{array} \right\}$$

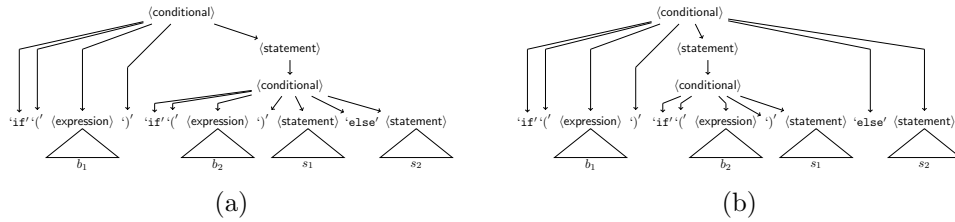
$L(G_1) = L(G_2)$ であり、 G_2 は曖昧ではない。これは、 and を or より優先するために、新たな非終端記号 A を導入して、 and が解析木で or よりも葉に近いように解釈されるようにしたためである。 G_2 では、単一生成規則 (single production) “ $E \rightarrow A$ ” を導入したため解析木のサイズが増大する。 n レベルの優先度の導入のために n 個の非終端記号が導入され、 n 個の単一生成規則が必要になる。曖昧性を除去するトレードオフとして、解析木が大きくなり、解析により長い時間がかかることになる。



このため、生成規則の適用順序を決めて解析の曖昧性を除去する場合もある。C言語などの条件分岐は `else` 付きの場合と `if ~ then ~ else ~`, `else` なしの場合 `if ~ then ~` の場合がある。

$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \end{aligned}$$

b_1, b_2 を $\langle \text{expression} \rangle$ から導出されるトークン列, s_1, s_2 を $\langle \text{statement} \rangle$ から導出されるトークン列とする。 `if (  $b_1$  ) if (  $b_2$  )  $s_1$  else  $s_2$` に対して図??のような2つの解析木が存在する。



C言語では、`else` は最も近い条件に対するものであると解釈するので、解析木として (b) であると決め、曖昧さを除去している。したがって、 b_1 が真で、 b_2 が偽であると評価された場合に実行される。構文解析における曖昧性は望ましいものとは言えないので、プログラム言語を定義する場合は無曖昧な CF 文法で定義することを基本とする。

上記の場合は、新たな予約語 `fi` を導入することで、曖昧さを解消できる。

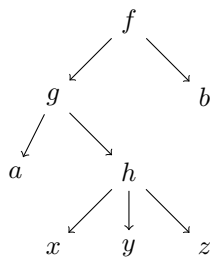
$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' fi' } \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \text{' fi' } \end{aligned}$$

4.1.4 解析木の表現

木の頂点の出力次数がわかっているとき解析木をトラバースする方法に基づいて、木構造を1次元的に表現することができる。木 T の根頂点を $root(T)$ と書くことにする。 $root(T)$ の次数が n であるとき $root(T)$ からの子を根頂点とする木を左から $T_1, T_2, \dots, T_n$ とする。

- **プリオーダー** $root(T)$ の次に T_1 のプリオーダー系列 $pre(T_1)$ を連結し、その次に $pre(T_2)$ を連結し、というのを再帰的に T_n まで繰り返す.
- **インオーダー** T_1 のインオーダーの系列 $in(T_1)$ に $root(T)$ を連結し、その次に $in(T_2), \dots, in(T_n)$ を連結する.
- **ポストオーダー** T_1 のポストオーダー $post(T_1), \dots, post(T_n)$ を連結し、最後に $root(T)$ を連結する.

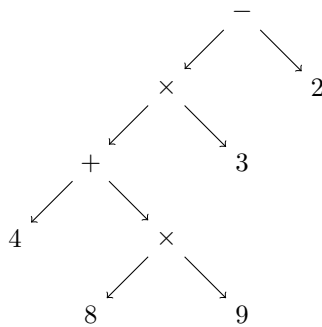
関数を表す式 $f(g(a, h(x, y, z)), b)$ に対して、関数名とその引数を親子関係として木構造で表すと、以下ようになる.



この木構造はプリオーダーで表すと $fgahxyzb$ となる.

f が 2 引数, g が 2 引数, h が 3 引数であることが既知であれば、この系列から関数の形を再現できる.

四則演算の算術式 $(4 + 8 \times 9) \times 3 - 2$ は以下のような構造を持つ.



これをポストオーダー系列で表すと $489 \times +3 \times 2-$ となる.

この記法は**逆ポーランド記法**と呼ばれる. それぞれの演算の引数が決まっていればスタックに対するルールに従って計算すれば式の評価が行われる.

逆ポーランド記法における評価の方法は以下のとおりである.

1. 数字がきたらスタックに積む
2. 演算がきたら引数の数だけスタックからポップして演算を施し、その結果をスタックにプッシュする.
3. 入力を全部読んだあとにスタックトップのあるのが評価値

$489 \times +3 \times 2-$ をスタックと対にしてこの方法で評価すると以下ようになる.

$$\begin{aligned}
(489 \times +3 \times 2-, []) &\rightarrow (89 \times +3 \times 2-, [4]) \\
&\rightarrow (9 \times +3 \times 2-, [4, 8]) \\
&\rightarrow (\times +3 \times 2-, [4, 8, 9]) \\
&\rightarrow (+3 \times 2-, [4, 72]) \\
&\rightarrow (3 \times 2-, [76]) \\
&\rightarrow (\times 2-, [76, 3]) \\
&\rightarrow (2-, [228]) \\
&\rightarrow (-, [228, 2]) \\
&\rightarrow (\epsilon, [226])
\end{aligned}$$

プリオーダーは、木の根を開始記号とする最左導出の規則適用に相当し、ポストオーダーは、木の根を開始記号とする最右導出の逆順の規則適用に相当する。

4.1.5 First と Follow

入力系列 w が与えられたとき、できるだけ効率的に CF 文法に従って $w \in L(G)$ (あるいは $w \notin L(G)$) を示したい。すなわち、 $S \Rightarrow^* w$ となる導出を示す必要がある。この際に導出に使う生成規則を選択する必要がある。例えば、 $A \rightarrow aB, A \rightarrow bC$ という2つの規則があり、 $\beta A \gamma$ という文形式から次の導出を行う。このとき、両方の生成規則が適用可能である。簡単のために最左導出であると考え、 $\beta \in \Sigma^*$ であり、入力文字列 w の接頭語が βa であるならば $A \rightarrow aB$ を選択すべきであり、接頭語が βb であるならば、 $A \rightarrow bC$ を選択すべきである。

このように次に処理すべきトークン (上の場合は β の次のトークン) によって生成規則の選択を決めることができる場合がある。

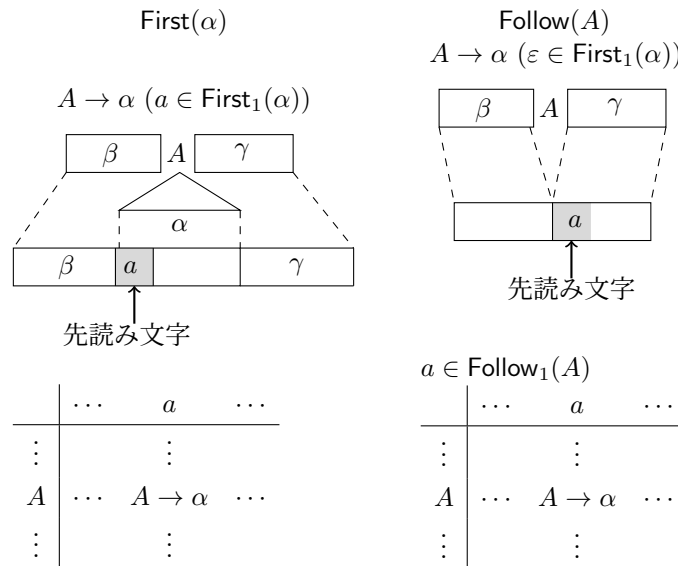


図 4.2: 導出の際の先読みによる決定

さらに、 $A \rightarrow \varepsilon$ の場合は、 A に続く文字が何かによってこの生成規則を適用できるかどうかを決めることができる場合がある。

コンパイラの構文解析においては、このようなまだ読み込んでいない**先読み文字**の情報をを用いて、生成規則の選択を決めることが行われる。もしこのような先読み文字の情報をを用いない場合は、すべての選択を試してみることになる。生成規則の選択がまちがっていた場合は、まちがいをやりなおす**バックトラック**によって規則を選び直さないといけなくなる。 βb が接頭語であるにもかかわらず、 $A \rightarrow aB$ で書き換えた場合は、書き換えをやりなおして、 $A \rightarrow bC$ に書き換える。このとき一旦読んでしまったトークンをまた入力系列に戻す。このような操作には時間がかかるため、一般的なプログラミング言語の設計においては生成規則の解析と先読み文字によって、適用できる生成規則が一意的になるような CF 文法で設計される。

生成規則が適用されるときに生成される接頭語を First、生成規則が適用されて生成される文字列の直後の接頭語を Follow といい、これらは生成規則をあらかじめ解析することによって得られる。

定義 4.2. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ が与えられたとき、系列 $\alpha \in (N \cup \Sigma)^*$ に対する Follow_1 集合を以下のように定義する。

$$\alpha \Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\} \cup \{\varepsilon\}$$

$$\alpha \not\Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\}$$

$\text{First}_1(\alpha)$ は、 α から導出によって生成される記号列で先頭に出現する可能性のある終端記号の集合を表す。 α から ε が生成される、つまり、 α が導出によって消失しうる場合には、 $\text{First}_1(\alpha)$ には ε という特別な要素を含めることにする。明らかに $\text{First}_1(\varepsilon) = \{\varepsilon\}$ である。

$\alpha \in (N \cup \Sigma)^+$ に対して、 α の最初の記号を $\text{fst}(\alpha)$ 、最初の記号を除いた系列を $\text{fol}(\alpha)$ と書くことにする。 $(\text{fst}(\alpha) \in N \cup \Sigma, \text{fol}(\alpha) \in (N \cup \Sigma)^* \text{ となる。})$ CF 文法 $\langle N, \Sigma, P, S \rangle$ において、 PO_P を以下のように定める。 $PO_P = \{\gamma \in (N \cup \Sigma)^* \mid \beta\gamma = \alpha, \beta \in (N \cup \Sigma)^*, A \rightarrow \alpha \in P\}$ PO_P は、生成規則 $A \rightarrow \alpha$ の右辺 α の接尾語の集合を表す。

$\alpha \in PO_P \cup N$ に対して $\text{First}_1(\alpha)$ は、以下のようにして計算することができる。

アルゴリズム 4.1. (First_1 を求めるアルゴリズム):

ステップ 1: $\text{First}_1(\varepsilon) \leftarrow \{\varepsilon\}$

ステップ 2: $\alpha \in PO_P \setminus \{\varepsilon\}$ に対して

(a) $\text{fst}(\alpha) \in \Sigma$ ならば、 $\text{First}_1(\alpha) \leftarrow \{\text{fst}(\alpha)\}$

(b) $\text{fst}(\alpha) \in N$ ならば、 $\text{First}_1(\alpha) \leftarrow \emptyset$

ステップ 3: すべての $\text{First}_1(\alpha)$ ($\alpha \in PO_P \cup N$) に新たな要素が追加されなくなるまで以下を繰り返す。

(a) $\text{fst}(\alpha) = A \in N$ である α に対して $\text{First}_1(\alpha)$ に以下の要素を追加する。

$\varepsilon \notin \text{First}_1(A)$ のとき、 $\text{First}_1(A)$

$\varepsilon \in \text{First}_1(A)$ のとき、 $\text{First}_1(A) \setminus \{\varepsilon\} \cup \text{First}_1(\text{fol}(\alpha))$

(b) $A \rightarrow \beta \in P$ に対して $\text{First}_1(\beta)$

定義 4.3. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して, $A \in N$ に対する $\text{Follow}_1(A)$ は以下のように定義される.

$$\begin{aligned} \text{Follow}_1(A) = & \{a \in \Sigma \mid S \Rightarrow^* \beta A a \gamma, \beta, \gamma \in (N \cup \Sigma)^*\} \\ & \cup \{\# \mid S \Rightarrow^* \beta A, \beta \in (N \cup \Sigma)^*\} \end{aligned}$$

ここで, $\#$ は入力の終わりを示す記号である.

アルゴリズム 4.2.

ステップ 1: $\text{Follow}_1(S) \leftarrow \{\#\}$

ステップ 2: $A \in N \setminus \{S\}$ に対して $\text{Follow}_1(A) \leftarrow \emptyset$

ステップ 3: $B \rightarrow \beta C \gamma \in P$ を満たす $B, C \in N$, $\beta, \gamma \in (N \cup \Sigma)^*$ に対してすべての $\text{Follow}_1(A) (A \in N)$ に新たな追加がなくなるまで以下を追加する.

(a) $\varepsilon \notin \text{First}_1(\gamma)$ のとき, $\text{First}_1(\gamma)$

(b) $\varepsilon \in \text{First}_1(\gamma)$ のとき, $\text{First}_1(\gamma) \setminus \{\varepsilon\} \cup \text{Follow}_1(B)$

4.2 下降型構文解析

4.2.1 左再帰性

手続き宣言の BNF の一部を以下に再掲する.

$$\begin{aligned} \langle \text{proc_decl} \rangle &::= \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' '}' ':' \langle \text{inblock} \rangle \\ \langle \text{id_list} \rangle &::= \langle \text{id_list} \rangle \text{' ' 'IDENT'} \\ &\quad | \text{'IDENT'} \end{aligned}$$

$\langle \text{proc_decl} \rangle$ からの最左導出の第一段階は $\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' '}' ':' \langle \text{inblock} \rangle$ であり, 次に $\langle \text{id_list} \rangle$ に対して導出が適用される. ここで, $\langle \text{id_list} \rangle ::= \langle \text{id_list} \rangle \text{' ' 'IDENT'}$ は左再帰的 (left-recursive) と呼ばれる. 右辺の最初に左辺と同じ非終端記号を含むので

$$\begin{aligned} \langle \text{id_list} \rangle &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' ' 'IDENT'} \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' ' 'IDENT'} \text{' ' 'IDENT'} \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' ' 'IDENT'} \text{' ' 'IDENT'} \text{' ' 'IDENT'} \\ &\quad \vdots \end{aligned}$$

のように同じルールが際限なく適用されて $\langle \text{id_list} \rangle$ が消えない可能性がある. $\langle \text{id_list} \rangle \rightarrow \text{'IDENT'}$ を適用すればこの導出を止めることができるが, $\text{'IDENT'} \in \text{First}_1(\langle \text{id_list} \rangle)$ であるので, 文形式の左端によっては別の規則を選択することができない.

左再帰性除去

$\{A \rightarrow A\alpha, A \rightarrow \beta\}$ という左再帰の規則を含む生成規則に対して、以下の生成規則集合は A から同じ文形式への導出を持ち、左再帰性を持つ規則を含まない。

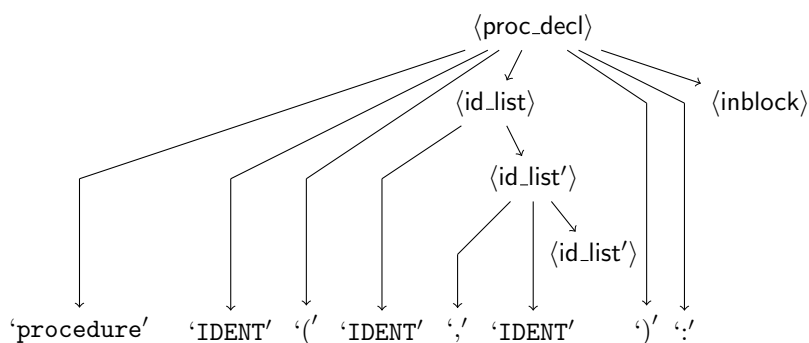
$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \\ A' &\rightarrow \varepsilon \end{aligned}$$

A' は新たな非終端記号である。

これに従って $\langle \text{id_list} \rangle$ に対して左再帰性を除去すると以下のようになる。

$$\begin{aligned} \langle \text{id_list} \rangle &::= \text{'IDENT' } \langle \text{id_list}' \rangle \\ \langle \text{id_list}' \rangle &::= \text{' , 'IDENT' } \langle \text{id_list}' \rangle \\ &\quad | \quad \varepsilon \end{aligned}$$

この場合の解析木は以下のようになる。



例 4.3. 加算 $+$ と乗算 $*$ を持つ式に対して、左再帰性をもつ生成規則と E からの導出が等しい左再帰性を除去した生成規則は以下のようになる。

$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{'IDENT' } \end{aligned}$	$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{'IDENT' } \end{aligned}$
(a) 左再帰性を持つ生成規則	(b) 左再帰性を除去した生成規則

4.2.2 左括り出し

以下の BNF 定義において

$$\begin{aligned} \langle \text{conditional} \rangle &::= \text{'if' } \langle \text{expression} \rangle \text{'then' } \langle \text{statement} \rangle \text{'else' } \langle \text{statement} \rangle \text{'fi' } \\ &\quad | \quad \text{'if' } \langle \text{expression} \rangle \text{'then' } \langle \text{statement} \rangle \text{'fi' } \end{aligned}$$

予約語 ‘if’ を見ただけではどちらの規則を適用すればよいかわからない。さらに新たな非終端記号 $\langle \text{conditional} \rangle$ を導入し、

$$\begin{aligned} \langle \text{conditional} \rangle &::= \text{‘if’} \langle \text{expression} \rangle \text{‘then’} \langle \text{statement} \rangle \langle \text{conditional} \rangle \\ \langle \text{conditional} \rangle &::= \text{‘else’} \langle \text{statement} \rangle \text{‘fi’} \\ &\quad | \text{‘fi’} \end{aligned}$$

とすれば、 $\langle \text{conditional} \rangle$ の規則は単一になり、 $\langle \text{conditional} \rangle$ についても先読みトークンが ‘else’ か ‘fi’ かで決定的に規則を選択できる。

同じ左辺を持つ生成規則において、右辺の共通な左端をまとめることを**左括り出し** (left-factoring) という。一般には $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ に対して、新たな非終端記号 A' を導入して $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ に変換することである。

4.2.3 LL 構文解析

必要に応じて左再帰性の除去、左括りだしを施した後、最左導出によって構文解析を行う。入力系列を先頭から読みながら最左導出を適用する構文解析は **LL (Left-to-right leftmost) 構文解析** と呼ばれる。

下向き構文解析表

最左導出において、開始記号から導出される文形式で最左に非終端記号 A を含む文形式は $\beta A \gamma$ の形であり、 $\beta \in \Sigma^*$, $\gamma \in (N \cup \Sigma)^*$ である。CF 文法 G の生成規則のうち、左辺に A を持つ生成規則が以下のようにになっているとする。

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

入力系列 w に対する解析木を構成する場合、 $w = \beta\beta'$ であるとき $A \rightarrow \alpha_i$ による $\beta A \gamma$ の導出が w に到達するためには、以下の条件が成り立つ必要がある。

- $\beta' \neq \varepsilon$ かつ $\varepsilon \notin \text{First}_1(\alpha_i)$ のとき $\text{fst}(\beta') \in \text{First}_1(\alpha_i)$
- $\beta' \neq \varepsilon$ かつ $\varepsilon \in \text{First}_1(\alpha_i)$ のとき、 $\text{fst}(\beta') \in \text{Follow}_1(A)$
- $\beta' = \varepsilon$ のとき、 $\alpha_i = \varepsilon$ かつ $\# \in \text{Follow}_1(A)$

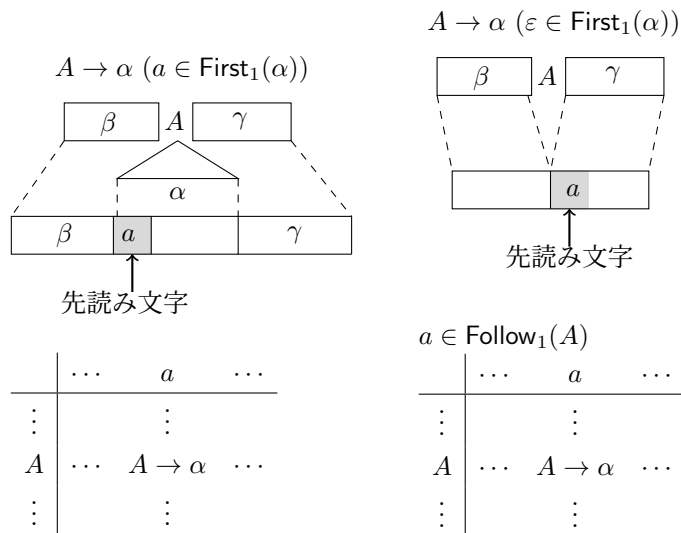
最左導出に対する下向き構文解析表は、縦向きに非終端記号ごとの行、横向きに $\Sigma \cup \{\#\}$ ごとの列を持つ表であり、以下のように構成される。

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

であるとき、

- $a \in \text{First}_1(\alpha_i) \cap \Sigma$ ならば、 A の行、 a の列に $A \rightarrow \alpha_i$ を追加する。

- $\varepsilon \in \text{First}_1(\alpha_i)$ ならば、 A の行、 $a \in \text{Follow}_1(A)$ の列に $A \rightarrow \alpha_i$ を追加



LL(1) 構文解析

定義 4.4. CF 文法 G に対する下向き構文解析表において、表のエントリの生成規則が高々 1 つであるとき、 G は **LL(1) 構文解析可能**である。このとき、 G は **LL(1) 文法**である。

下向き構文解析表は、最左導出可能なすべての生成規則を数え上げる。したがって、CF 文法 G が LL(1) 構文解析可能であるとき、 $w \in L(G)$ であるとき、かつ、そのときに限り、開始記号から w への決定的な生成規則の適用によって導出が存在する。明らかに下向き構文解析表以外の生成規則を適用しても導出を構成することはできない。

LL 構文解析は、構文木を最左導出で根方向から葉方向に構成する。先読み文字（トークン）以外の入力文字を直接参照することなく、適用可能な生成規則を正確に予測しながら構成するため**予測的構文解析**とも呼ばれる。

先読み文字を k 文字読んで導出の生成規則を決定できる CF 文法を LL(k) 文法という。2 文字以上先読みすると一般にはより適用する生成規則を限定することができる。しかし、一般的には 2 以上の k が使われることは少ない。

4.2.4 拡張 CF 文法と ELL(1) 文法

拡張 CF 文法は、生成規則の右辺に正規表現の選択と Kleene 閉包を導入して拡張した CF 文法である。この拡張によって CF 文法の表現力が増加することはない。この拡張によってプログラミング言語をよりコンパクトに定義することができるようになる。

アルゴリズム 4.3. 拡張 CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して等価な CF 文法 $G' = \langle N', \Sigma', P', S' \rangle$ を以下のようにして構成できる。 P のすべての右辺が $(N \cup \Sigma)^*$ の要素になるまでステップ 1 を繰り返す。

ステップ 1: $A \rightarrow e \in P$ のうち $e \notin (N \cup \Sigma)^*$ である生成規則を P から取り除く.

(a) $e = e_1 | e_2$ の場合: P に $A \rightarrow e_1$ と $A \rightarrow e_2$ に加える.

(b) $e = e_1 e_2$ の場合: 新たな非終端記号 B を導入し, $A \rightarrow e_1 B$ と $B \rightarrow e_2$ を P に加え, N に B を追加する.

(c) $e = e_1^*$ の場合: 新たな非終端記号 B を導入し, $A \rightarrow B$, $B \rightarrow e_1 B$, $B \rightarrow \varepsilon$ を P に追加し, N に B を追加する.

最終的に得られた N を N' とし, P を P' とする.

変換に基づいて得られる CF 文法に対して $\text{First}_1(\alpha)$ を求めることができる. この First_1 を用いて拡張 CF 文法に対する First_1 を構成できる.

定義 4.5. 拡張 CF 文法の生成規則 $A \rightarrow e$ に対して

- $e = \varepsilon$ のとき, $\text{First}_1(e) = \{\varepsilon\}$
- $e \in \Sigma$ のとき, $\text{First}_1(e) = \{e\}$
- $e = e_1 | e_2$ のとき, $\text{First}_1(e) = \text{First}_1(e_1) \cup \text{First}_1(e_2)$
- $e = e_1 e_2$ のとき,
 - $\varepsilon \in \text{First}_1(e_1)$ のとき, $\text{First}_1(e) = \text{First}_1(e_1) \setminus \{\varepsilon\} \cup \text{First}_1(e_2)$
 - $\varepsilon \notin \text{First}_1(e_1)$ のとき, $\text{First}_1(e) = \text{First}_1(e_1)$
- $e = e_1^*$ のとき, $\{\varepsilon\} \cup \text{First}_1(e_1)$

定義 4.6. $A \rightarrow e \in P$ のとき, $\text{Follow}_1(e) = \text{Follow}_1(A)$

$e = e_1 | e_2$ のとき, $\text{Follow}_1(e_1) = \text{Follow}_1(e)$, $\text{Follow}_1(e_2) = \text{Follow}_1(e)$

$e = e_1 e_2$ のとき, $\text{Follow}_1(e_2) = \text{Follow}_1(e)$

$\varepsilon \in \text{First}_1(e_2)$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_2) \setminus \{\varepsilon\} \cup \text{Follow}_1(e)$

$\varepsilon \notin \text{First}_1(e_2)$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_2)$

$e = e_1^*$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_1) \setminus \{\varepsilon\} \cup \text{Follow}_1(e)$

Follow₁ は大きな項からその部分項へ計算される.

拡張 CF 文法に対しても CF 文法と同様に下向き構文解析表を構成することができる.

定義 4.7. 拡張 CF 文法 G に対する下向き構文解析表を構成したとき, 表の各エントリーの生成規則が高々 1 つであるとき, $ELL(1)$ 構文解析可能である. このとき, G は $ELL(1)$ 文法である.

4.2.5 JJTree による降下型構文解析プログラムの生成

JJTree は、JavaCC のプリプロセッサである。jjt ファイルには、生成規則と字句解析の仕様を記述する。

JJTree のプログラムには、字句解析に用いるトークンの定義と生成規則を表すクラスメソッドを記述する。例えば、 $E \rightarrow T(+T)^*$ といった生成規則は以下のようなクラスメソッドとして記述する。

```
void E() :  
{  
  {  
    T()  
    ( <ADDOP> T() ) *  
  }  
}
```

ここで、<ADDOP> は+のトークンである。生成規則の左辺をクラスメソッドとし、本体を右辺として記述する。JJTree は、下降型構文解析プログラムを生成する。

JJTree によるプログラム生成の流れを図??に示す。jjt ファイルを JJTree プロセッサに入力し、javacc ファイル (.jj ファイル) といくつかの java ファイルが生成される。さらに javacc で変換し、すべてのファイルを java ファイルに変換する。

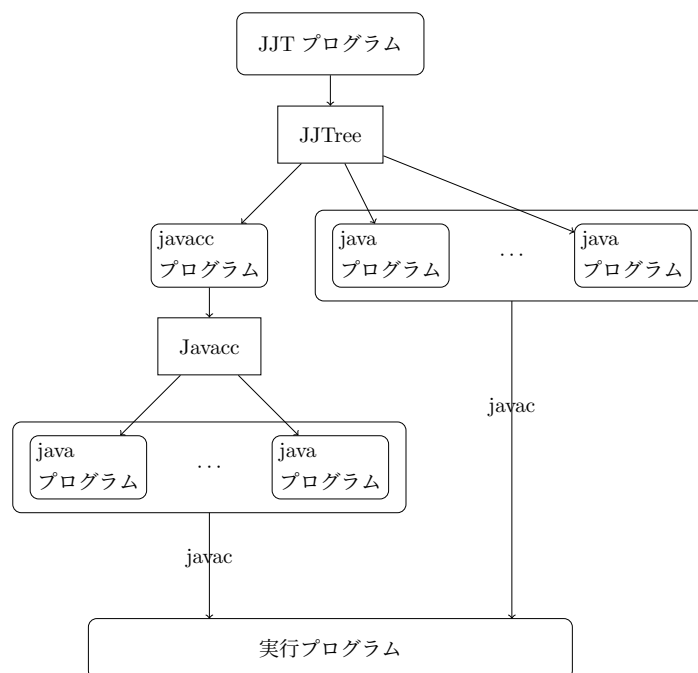


図 4.3: jjtree-config

4.2.6 例

(本内容の javacc コードは、早乙女健治 (著) : Javacc : コンパイラ・コンパイラ for Java (テクノプレス刊 2003) からの引用によるものです.)

以下の拡張 CF 文法で定義された四則演算式を構文解析する.

$$\begin{aligned}\langle \text{AddExpr} \rangle &::= \langle \text{MulExpr} \rangle ((+|-) \langle \text{MulExpr} \rangle) * \\ \langle \text{MulExpr} \rangle &::= \langle \text{PrimExpr} \rangle ((*|/) \langle \text{PrimExpr} \rangle) * \\ \langle \text{PrimExpr} \rangle &::= '(' \langle \text{AddExpr} \rangle ')' | \text{'Literal'}\end{aligned}$$

```
/** simple calculator */
options {
    STATIC=false;
    MULTI=true;
}
PARSER_BEGIN(Calc)
public class Calc {
    public static void main(String args[]) throws ParseException {
        while (true) {
            Calc parser = new Calc(System.in);
            switch (parser.CalcUnit()) {
                case -1:
                    return;
                case 0:
                    continue;
                default:
                    //      ((SimpleNode)parser.jjttree.rootNode()).dump("");
                    parser.jjttree.rootNode().calculate();
            }
        }
    }
}
PARSER_END(Calc)
SKIP:
{
    " " | "\r"
}

TOKEN:
{
    < ADDOP: "+" >
```

```

| < SUBOP: "-" >
| < MULOP: "*" >
| < DIVOP: "/" >
| < LP: "(" >
| < RP: ")" >
| < LITERAL: (["0"-"9"])* >
| < NL: "\n" >
}

int CalcUnit() :
{}
{
    ( AddExpr() <NL>
      { return 1; }
    | <NL>
      { return 0; }
    | <EOF>
      { return -1; }
    )
}

void AddExpr() #void :
{}
{
    MulExpr()
    (
        <ADDOP> MulExpr() #AddNode(2)
    | <SUBOP> MulExpr() #SubNode(2)
    )*
}

void MulExpr() #void :
{}
{
    PrimExpr()
    (
        <MULOP> PrimExpr() #MulNode(2)
    | <DIVOP> PrimExpr() #DivNode(2)
    )*
}

void PrimExpr() #void :
{}

```

```

{
    Literal()
    | <LP> AddExpr() <RP>
}
void Literal() :
{ Token t; }
{
    t=<LITERAL>
    { jjtThis.litValue = Integer.parseInt(t.image); }
}

```

AddExpr() 内の, #AddNode(2), #SubNode(2) MulExpr() 内の, #MulNode(2), #DivNode(2) はそれぞれ, 四則演算子が2項演算なので, 2つノードが生成されれば, 新たにノードを生成する効果を持つ。

4.3 上昇型構文解析

下降型構文解析は単純で効率的な構文解析手法ではあるが, CF 文法が実用的には LL(1) 文法である必要がある, LL(1) 言語に用いることのできる CF 文法の生成規則の適用と意図する構文木の対応が必ずしも明確ではないなどの特徴があるため, 比較的シンプルで構文と意味の間に柔軟性がある言語の解析に向いている。下降型では, 入力系列を読み取ることなく, 構文木の根の方向から適用する生成規則を限られた先読み文字によって決定することができなければならない。

これに対して, 生成規則の右辺を入力および入力から導出の逆動作 (還元) で得られた文形式に対してマッチングすることで, 左辺に置き換える還元を行って, 開始記号まで還元することによって, 逆方向から導出を構成する構文解析手法がある。これは, 構文木の葉の方向から根を構成するので**上昇型構文解析**と呼ばれる。ここでは, 上昇型構文解析の計算機構であるプッシュダウン変換機械 (pushdown transducer) を示し, この上での上昇型構文解析として LR 構文解析を示す。一般に用いられる LR(1) 構文解析は LL(1) 構文解析よりも適用できる CF 文法の範囲が大きい。Yacc などの構文解析プログラム生成ツールが対象としている LALR(1) 構文解析は LL(1) 構文解析とは適用できる CF 文法の範囲は比較できないが, 左括りだしや左再帰性の除去などが必要なく, 構文木を直接的に得られるためよく用いられる。LR 構文解析として, SLR(1) 構文解析手法を示したのち, その拡張として LR(1) 構文解析手法を示す。

4.3.1 プッシュダウン変換機械

プッシュダウン変換機械 (pushdown transducer, PDT) は, プッシュダウンオートマトンに出力テープを追加した計算モデルである。プッシュダウンオートマトン (PDA) はスタックへのプッシュ, ポップ動作によってスタック記号を容量制限のない記憶に格納することができる。プログラミング言語は文脈自由文法で定義されることから, その受理にはプッシュダウンオートマトンを用いる必

要がある．さらに構文解析の際にはどの生成規則が使われたかを出力する必要があるので，出力テープを持つ PDT を用いる．下降型構文解析においては，入れ子構造をもつ手続き呼び出しがスタックの働きをしている．上昇型構文解析では，構文解析表に従って動作する PDT を構成する．

定義 4.8. PDT を $T = \langle \Gamma, \Sigma, O, \Delta, I_0, F \rangle$ として定義する．ここで

Γ スタック記号の有限集合

Σ 入力記号の有限集合

O 出力記号の有限集合

Δ 遷移関数 $\Gamma^+ \times \Sigma \rightarrow \Gamma^* \times O^* \times \{0, 1\}$

$I_0 \in \Gamma$ 初期スタック記号

$F \subseteq \Gamma \cup \{\varepsilon\}$ 最終スタック記号の集合

Σ には特別の記号 $\#$ を含む． $\#$ は入力テープの右端を示す希望である． (s, i, o) の 3 項組を時点表示 (configuration) という．スタック $s \in \Gamma^*$ はスタック記号の有限系列，入力テープ $I \in \Sigma^*$ ，出力テープ $o \in O^*$ とする． s は左側がスタックの底，右側がスタックトップを表す．遷移関数 $\Delta : \Gamma^+ \times \Sigma \rightarrow \Gamma^* \times O^* \times \{0, 1\}$ によって時点表示を更新する．

- $\Delta(u, a) = (w, \pi, 1)$ のとき，遷移を $u \xrightarrow{a} (w, \pi)$ と書き，PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, x, o\pi)$ で表す．
- $\Delta(u, a) = (w, \pi, 0)$ のとき，遷移を $u \xrightarrow{(a)} (w, \pi)$ と書き，PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, ax, o\pi)$ で表す．

$u \xrightarrow{a} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには， T のスタックの上部 u を w に置き換え，出力テープに π を追加して， $o\pi$ とする． $u \xrightarrow{(a)} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには，スタックの上部 u を v に置き換えて，出力テープに π を追加して $o\pi$ とする．このとき，入力 a は読み込まない．遷移を行うためには入力テープの先頭は a である必要があるが，入力テープのポインタは動かさない．この動作を先読みといい， a を先読み記号という．

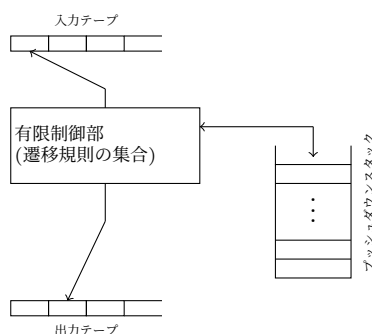


図 4.4: プッシュダウン変換機械 (PDT)

4.3.2 LR 構文解析

CF 文法の上昇型構文解析器 (bottom-up parser) は，入力として $x\# \in \Sigma^*$ を受取り， $x\#$ から導出の逆関係 $\Leftarrow$ を用いて，入力の左側から還元 (reduction) により開始記号 S を得る．

$$w = \alpha_n \Leftarrow \alpha_{n-1} \Leftarrow \cdots \Leftarrow \alpha_1 \Leftarrow \alpha_0 = S$$

ここで, $\beta\alpha\gamma \Leftarrow \beta A\gamma$ となる生成規則 $A \rightarrow \alpha$ が存在する. α_0 においては, $\beta = \varepsilon$ である. 上記の還元系列は, 逆方向から見ると最右導出になっている. 例えば $\alpha_1 = AB \Leftarrow S$ であるとき, $\alpha_2 = A\beta \Leftarrow AB$ である. $A \rightarrow \alpha$ による還元が $\alpha_{i+1} = \beta\alpha\gamma \Leftarrow \beta A\gamma = \alpha_i$ であるとき, α を α_{i+1} のハンドル (handle) という. α_i から決定的にハンドルを見つけて還元することが効率的な構文解析を構成するために必要である. α_i におけるハンドルが α である場合, $\alpha_i = \beta\alpha\gamma$, $\beta \in \Sigma^*$ であり, β は入力 w の接頭語になっている.

上昇型構文解析器を構成するとき, CF 文法 $G = \langle N, \Sigma, P, S \rangle$ から**拡大文法** (augmented grammar) $G' = \langle N \cup \{S'\}, \Sigma \cup \{\#\}, P \cup \{S' \rightarrow S\# \}, S' \rangle$ を構成する. ここで, S' は新たな非終端記号であり, $\#$ は終端記号である. 拡大文法における上昇型構文解析では $S' \rightarrow S\#$ による還元か, ハンドルが見つからない場合に終了する. 前者の場合, 最右導出が構成できて $w \in L(G)$ が示され, 後者の場合は, $w \notin L(G)$ であり, 構文エラーである.

例 4.4. 文法 $\langle \{i, n, +, *, (,)\}, \{E\}, P, E \rangle$ に対する拡大文法は $\langle \{i, n, +, *, \#, (,)\}, \{E, S'\}, P \cup \{S' \rightarrow E\# \}, S' \rangle$ となる. ここで,

$$P = \left\{ \begin{array}{ll} 0: S' \rightarrow E\# & 3: E \rightarrow (E) \\ 1: E \rightarrow E + E & 4: E \rightarrow i \\ 2: E \rightarrow E * E & 5: E \rightarrow n \end{array} \right\}$$

入力系列を $(i+n)+i*i\#$ とする.

$$\begin{aligned} S' &\Rightarrow_{rm} E\# \\ &\Rightarrow_{rm} E + E\# \\ &\Rightarrow_{rm} E + E * E\# \\ &\Rightarrow_{rm} E + E * i\# \\ &\Rightarrow_{rm} E + i * i\# \\ &\Rightarrow_{rm} (E) + i * i\# \\ &\Rightarrow_{rm} (E + E) + i * i\# \\ &\Rightarrow_{rm} (E + n) + i * i\# \\ &\Rightarrow_{rm} (i + n) + i * i\# \end{aligned}$$

これを還元によって受理する PDT の動作は以下ようになる.

$(I_0, (i+n)+i*i\#, \varepsilon)$		
$\vdash$	$(I_0 (,$	$i+n)+i*i\#, \varepsilon)$
$\vdash$	$(I_0 (i,$	$+n)+i*i\#, \varepsilon)$
$\vdash$	$(I_0 (E,$	$+n)+i*i\#, 4)$
$\vdash$	$(I_0 (E+,$	$n)+i*i\#, 4)$
$\vdash$	$(I_0 (E+n,$	$) + i*i\#, 4)$
$\vdash$	$(I_0 (E+E,$	$) + i*i\#, 45)$
$\vdash$	$(I_0 (E+E),$	$+i*i\#, 45)$
$\vdash$	$(I_0 (E),$	$+i*i\#, 451)$
$\vdash$	$(I_0 E,$	$+i*i\#, 4513)$
$\vdash$	$(I_0 E+,$	$i*i\#, 4513)$
$\vdash$	$(I_0 E+i,$	$*i\#, 4513)$
$\vdash$	$(I_0 E+E,$	$*i\#, 45134)$
$\vdash$	$(I_0 E+E*,$	$i\#, 45134)$
$\vdash$	$(I_0 E+E*i,$	$\#, 45134)$
$\vdash$	$(I_0 E+E*E,$	$\#, 451344)$
$\vdash$	$(I_0 E+E,$	$\#, 4513442)$
$\vdash$	$(I_0 E,$	$\#, 45134421)$
$\vdash$	$(I_0 E\#,$	$\varepsilon, 45134421)$
$\vdash$	$(I_0 S',$	$\varepsilon, 451344210)$

定義 4.9. 最右導出 $S' \Rightarrow_{rm}^* \alpha A w \Rightarrow_{rm} \alpha \beta w$ (ここで $w \in \Sigma\{\#\}$) において, αA の接頭語および $\alpha \beta$ の接頭語を G' の**可延頭部** (*viable prefix*) という.

可延頭部は, 上昇型構文解析において構文解析が進んでいる状態を示している.